Chapter Seven
Test Automation

Introduction to Test Automation

- Test Automation is the use of software tools and scripts to execute tests automatically, compare actual results with expected results, and report outcomes without manual intervention.
- Instead of a tester repeatedly performing the same actions, automation tools can run the tests faster, more accurately, and more consistently.
- Example:
  - Instead of manually logging into a website every day to check if login works, an automated test script can do it in seconds.

Why Test Automation is Important

- Automation is important because it helps in:
  - Speed
    - Automated tests run much faster than manual testing.
  - Accuracy
    - Automation reduces human errors.
  - Reusability
    - Once a test script is created, it can be reused many times.
  - Cost Saving (Long-term)
    - Although initial setup may be expensive, automation saves time and money over repeated use.
  - Continuous Testing
    - Supports DevOps and CI/CD pipelines where testing happens after every code change.

What to Automate

- Not every software test should be automated. While automation provides speed, consistency, and efficiency, some tests still require human judgment, creativity, and observation.
- Therefore, organizations must carefully decide which test cases are most suitable for automation and which are better performed manually.

Suitable Tests for Automation

- Repetitive Tests
  - Repetitive tests are tests that must be performed again and again whenever a new version of the software is released.
  - Manually repeating the same test steps each time can be tiring, slow, and error-prone.
  - Automation is highly useful in such situations because the script can repeat the same process consistently without mistakes.

- Regression Testing
  - Regression testing is performed after developers modify the software to ensure that new changes have not broken existing features.
  - As software grows, regression testing becomes larger and more difficult to perform manually.
  - Automation is excellent for regression testing because the same test suite can be rerun after every code change.

- Smoke Testing
  - Smoke testing is a quick set of basic tests performed after a new software build or deployment.
  - Its purpose is to verify that the main functions of the application are working and that the build is stable enough for further testing.
  - Since smoke tests are short, repetitive, and needed regularly, they are ideal for automation.

- Data-Driven Tests
  - Data-driven testing means running the same test with different sets of input data.
  - Instead of creating many separate tests manually, one automated script can run multiple combinations of inputs.
  - This is useful for forms, calculators, login pages, and validation systems.

- Performance Testing
  - Performance testing measures how well a system performs under normal or heavy usage conditions.
  - It checks speed, response time, stability, scalability, and stress tolerance.
  - These tests require simulation of many users or large workloads, which is difficult to do manually. Automation tools are necessary.

- Cross-Browser Testing
  - Web applications may behave differently in different browsers or devices. Cross-browser testing checks whether the website works correctly on multiple browsers such as Chrome, Firefox, Safari, and Edge.
  - Running the same tests manually on every browser can be time-consuming.
  - Automation tools can execute the same scripts across browsers.

Activity

- Decide What to Automate

Scenario
1. Testing login after every software release
2. Checking if website colors are attractive
3. Testing 100 password combinations on a login form
4. Checking if 2,000 users can use the site at once
5. Exploring unexpected user behavior
6. Running tests on Chrome, Firefox, and Safari



Activity

- Decide What to Automate

Scenario	
1. Testing login after every software release	**A**
2. Checking if website colors are attractive	M
3. Testing 100 password combinations on a login form	A
4. Checking if 2,000 users can use the site at once	A
5. Exploring unexpected user behavior	M
6. Running tests on Chrome, Firefox, and Safari	A

Tests Not Ideal for Automation

- Although test automation provides many benefits such as speed, consistency, and efficiency, not every type of test is suitable for automation.
- Some tests require human thinking, observation, creativity, or frequent adaptation. In such cases, manual testing is often more effective and practical than automated testing.
- The following are tests that are generally not ideal for automation:

- Exploratory Testing
  - Exploratory testing is a type of testing where the tester actively explores the software without following a fixed script.
  - The tester learns about the system while testing it, tries unexpected actions, and searches for hidden defects.
  - This kind of testing depends heavily on human creativity, curiosity, intuition, and experience.
  - Automated tools follow predefined instructions, but they cannot think like a human or improvise new testing ideas during execution.

- Usability Testing
  - Usability testing checks how easy, clear, efficient, and satisfying the software is for users.
  - It focuses on user experience rather than only technical correctness.
  - Automation tools can verify if a button works, but they cannot truly judge whether the button is confusing, unattractive, hard to find, or frustrating to use.

- One-time Tests
  - One-time tests are tests that only need to be performed once or very rarely.
  - Creating automation scripts for such tests may take more time and cost than manually executing them.
  - Automation is most valuable when tests are repeated many times.
  - If the test is only needed once, writing, debugging, and maintaining a script may not be worthwhile.

- Frequently Changing Features
  - If a feature changes often, especially the user interface (UI), automation scripts can break repeatedly.
  - Buttons may move, field names may change, pages may be redesigned, or workflows may be updated every few days.
  - This creates high maintenance costs because testers must constantly rewrite scripts.

How to Automate Testing

1. **Select Test Cases:** Choose important and repetitive test cases.
2. **Choose Automation Tool**

Tool	Used For
Selenium	Web applications
Cypress	Modern web testing
Appium	Mobile apps
JUnit	Java unit testing
TestNG	Java testing framework
Postman	API testing
LoadRunner / JMeter	Performance testing

3. **Design Test Scripts:** Write scripts that perform actions.
4. **Execute Tests:** Run scripts manually or automatically.

Benefit and challenges of automation

Benefits	Challenges
Faster execution Better coverage Reusable scripts Early bug detection Supports CI/CD Reliable repeated testing	Initial cost is high Needs skilled testers Maintenance required Tool limitations Flaky tests may fail randomly

Automation Frameworks

- An automation framework is a structured set of guidelines, tools, coding standards, reusable components, and best practices used to design, develop, execute, and maintain automated tests efficiently.
- Instead of writing random test scripts one by one, a framework provides an organized structure. It helps teams create automation that is **scalable, reusable, maintainable, readable, and efficient**.
- Think of a framework as the **architecture of your test automation project**.

- Main Components of a Framework
  - Test scripts
  - Test data
  - Reusable functions
  - Configuration files
  - Object locators
  - Reports
  - Logs
  - Setup and teardown methods
  - Utilities (screenshots, waits, database checks)

Linear Framework

- The Linear Framework is the simplest type of automation framework.
- It follows a straight sequence of steps from beginning to end.
- It is often called Record and Playback Framework because tools record user actions and replay them later.
- **Best Use Case**
  - Small temporary projects
  - Demonstrations
  - Learning automation basics

Modular Framework

- In the Modular Framework, the application is divided into separate modules or components.
- Each module gets its own test scripts or reusable functions.
- Examples of modules:
  - Login module
  - Search module
  - Cart module
  - Payment module
- Then full test cases combine these modules.
- **Best Use Case**
  - Medium to large applications
  - Systems with repeated workflows

Data-Driven Framework

- In a Data-Driven Framework, test logic is separated from test data. The same script runs multiple times using different inputs stored externally.
- Data sources:
  - Excel files
  - CSV files
  - JSON
  - XML
  - Database
- **Best Use Case**
  - Forms
  - Login pages
  - Input validation
  - Calculators
  - APIs with many payload combinations

Keyword-Driven Framework

- In Keyword-Driven Framework, tests are created using predefined action words (keywords) rather than writing detailed code.
- Examples of keywords:
  - OPEN_BROWSER
  - LOGIN
  - CLICK
  - SEARCH
  - VERIFY_TEXT
  - LOGOUT
- These keywords are mapped to underlying automation functions.

Step	Keyword	Value
1	OPEN_BROWSER	Chrome
2	LOGIN	admin,1234
3	SEARCH	Laptop
4	VERIFY_TEXT	Results Found